Monad BFT

Security Assessment

December 31st, 2025 — Prepared by OtterSec

James Wang james.wang@osec.io

Jinwoo Lee jin@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MCS-ADV-00 | Panic due to Unchecked Timestamp Overflow 7

OS-MCS-ADV-01 | Message Rejection due to Oversized ConfirmGroup Peers 8

OS-MCS-ADV-02 | Single-Peer BlockSync Liveness Logic 10

OS-MCS-ADV-03 | Risk of Denial-of-Service via Validator Collusion 11

OS-MCS-ADV-04 | Range-Adjacency Check Off-By-One 12

OS-MCS-ADV-05 | Incorrect Group Expiration Check 13

OS-MCS-ADV-06 | Incomplete cache eviction 14

OS-MCS-ADV-07 | Improper Inactivated Symbol Handling 15

OS-MCS-ADV-08 | Epoch Boundary Timing Inconsistency 16

General Findings **17**

OS-MCS-SUG-00 | Validate TimeoutMessage with same round QC 18

OS-MCS-SUG-01 | UDP/TCP Channel Congestion 19

OS-MCS-SUG-02 | Missing Optional-Field Discriminators 20

OS-MCS-SUG-03 | TCP Message Drop via TOCTOU 21

OS-MCS-SUG-04 | Code Refactoring 23

Vulnerability Rating Scale **25**

Procedure **26**

© 2025 Otter Audits LLC. All Rights Reserved.

Executive Summary

Overview

Category Labs engaged OtterSec to assess the `consensus` and `raptorcast` programs. This assessment was conducted between October 29th and December 18th, 2025. For more information on our auditing methodology, refer to chapter 06.

Key Findings

We produced 14 findings throughout this audit engagement.

In particular, we identified several vulnerabilities, including one that enables anyone to craft an invalid udp handshake to crash validators nodes (OS-MCS-ADV-00).

Additionally, BlockSync selects only one peer per request, allowing an adversarial or slow peer to stall block retrieval and induce unnecessary timeouts (OS-MCS-ADV-02). Furthermore, epoch start is computed from the boundary block's proposal round, resulting in the new epoch to start in the past in case of delayed commitment, which creates validator readiness gaps and ambiguity over which validator set applies to certain rounds, complicating replay and potentially undermining safety at epoch boundaries (OS-MCS-ADV-08).

We also made recommendation for refactoring the codebase for better functionality and addressing inconsistencies (OS-MCS-SUG-04). We further advised explicitly adding optional-field discriminators, as optional block-header fields are currently encoded without explicit presence markers, which may result in missing earlier fields to misalign subsequent values and be decoded into incorrect slots (OS-MCS-SUG-02). Moreover, we highlighted a scenario where a brief TOCTOU window may result in TCP messages to be dropped when a connection task times out before new messages are pushed, leaving them unsent in a newly created channel (OS-MCS-SUG-03).

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Scope

The source code was delivered to us in a git repository at [https://github.com/category-labs/monad-bft. This audit was performed against commit dae0789.

A brief description of the programs is as follows:

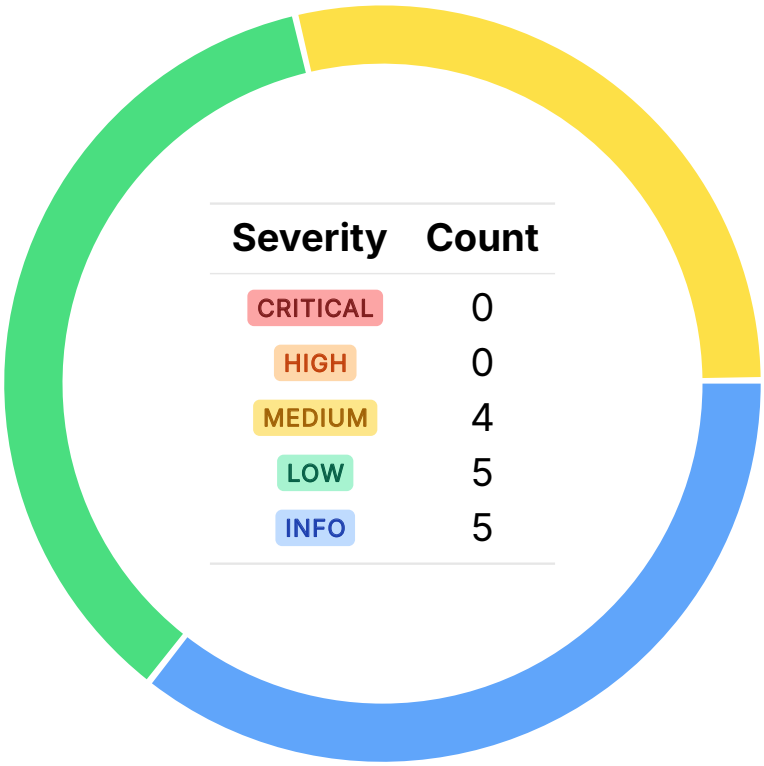
Name	Description
consensus	It orders transactions and runs a Byzantine fault-tolerant protocol to agree on and produce canonical blocks, which are written to a ledger. These finalized blocks are then consumed by the execution layer to update blockchain state deterministically.
raptorcast	An erasure-coded broadcast layer designed to disseminate large blockchain data efficiently.

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Findings

Overall, we reported 14 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	4
LOW	5
INFO	5

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in chapter 05.

ID	Severity	Status	Description
OS-MCS-ADV-00	MEDIUM	RESOLVED	An attacker may supply a crafted WireAuth handshake timestamp that overflows during `Tai64N::to_system_time`, triggering a panic when processing the initiator's first handshake message.
OS-MCS-ADV-01	MEDIUM	RESOLVED	The publisher utilizes an unbounded peer list, while full nodes strictly enforce the `max_group_size` constraint promised earlier during the `Prepare` phase, resulting in the compliant full nodes rejecting the message if asynchronous invite acceptances inflate `full_nodes_accepted` beyond the negotiated limit.
OS-MCS-ADV-02	MEDIUM	RESOLVED	BlockSync selects only one peer per request, allowing an adversarial or slow peer to stall block retrieval and induce unnecessary timeouts.
OS-MCS-ADV-03	MEDIUM	RESOLVED	Malicious validators may exploit the `max_num_group` limit to fill a full node's slots, preventing it from joining honest groups and creating a denial-of-service.
OS-MCS-ADV-04	LOW	RESOLVED	The adjacency check in `reorder_to_gso` is off by one because Rust ranges are half-open.

© 2025 Otter Audits LLC. All Rights Reserved.

OS-MCS-ADV-05	LOW	RESOLVED	The cleanup condition in `enter_round` treats `end_round` as inclusive, allowing groups that have already expired to remain in the schedule.
OS-MCS-ADV-06	LOW	RESOLVED	The cache may temporarily exceed its `max_total_size` because evictions are based on counts or expiration, not actual message sizes.
OS-MCS-ADV-07	LOW	RESOLVED	`try_inactive_gaussian` incorrectly returns false for weight-1 inactivated buffers, delaying symbol reactivation and stalling decoding progress.
OS-MCS-ADV-08	LOW	RESOLVED	Epoch start is computed from the boundary block's proposal round, not its commit round, so delayed commitment may result in the new epoch to start in the past. This creates validator readiness gaps and ambiguity over which validator set applies to certain rounds, complicating replay and potentially undermining safety at epoch boundaries.

© 2025 Otter Audits LLC. All Rights Reserved.

Panic due to Unchecked Timestamp Overflow MEDIUM OS-MCS-ADV-00

Description

`handshake::accept_handshake_init` correctly authenticates and decrypts the initiator's timestamp, but it assumes that any successfully parsed `Tai64N` value may be safely converted into `SystemTime`. However, `Tai64N::to_system_time` may overflow or underflow when the timestamp represents a time far outside the `SystemTime` representable range. This allows an attacker to craft a WireAuth `HandshakeInitiation` in which the decrypted `Tai64N` timestamp overflows during conversion to `SystemTime` via `to_system_time`, triggering a panic within `accept_handshake_init`.

>_ *monad-wireauth/src/protocol/handshake.rs* RUST

```rust
pub fn accept_handshake_init(
    responder_static_keypair: &monad_secp::KeyPair,
    msg: &mut HandshakeInitiation,
) -> Result<(HandshakeState, SystemTime), HandshakeError> {
    [...]
    let timestamp = Tai64N::from_slice(&msg.encrypted_timestamp).map_err(|_| {
        HandshakeError::InvalidTimestamp {
            size: msg.encrypted_timestamp.len(),
        }
    })?;
    let system_time = timestamp.to_system_time();

    Ok((state, system_time))
}
```

Remediation

Explicitly bound acceptable timestamp ranges before conversion.

Patch

Resolved in f9668de.

© 2025 Otter Audits LLC. All Rights Reserved.

Message Rejection due to Oversized ConfirmGroup Peers MEDIUM OS-MCS-ADV-01

Description

In `publisher::advance_invites`, when the publisher decides that the group formation phase is complete and constructs a `ConfirmGroup` message, it unconditionally includes all nodes currently present in `full_nodes_accepted.list`. However, the list may have grown beyond `max_group_size` because new `AcceptGroup` responses may arrive asynchronously after the publisher decides to confirm the group, but before `advance_invites` is invoked again. Because `advance_invites` is triggered only when the publisher advances rounds, its execution is not synchronized with the arrival of acceptance messages.

>_ *monad-bft/monad-raptorcast/src/raptorcast_secondary/publisher.rs* RUST

```rust
pub fn advance_invites(
    &mut self,
    curr_timestamp: TimePoint,
    cfg: &GroupSchedulingConfig,
    validator_id: NodeId<CertificateSignaturePubKey<ST>>,
) -> Option<(FullNodesGroupMessage<ST>, FullNodesST<ST>)> {
    [...]
    // group is starting soon
    {
        [...]
        self.next_invite_tp = TimePoint::MAX; // lock the group
        let confirm_data = ConfirmGroup {
            prepare: prep_grp_data,
            peers: self.full_nodes_accepted.list.clone(),
            name_records: Default::default(), // to be filled by next layer
        };
        // ConfirmGroup is sent to all accepted peers
        let grp_msg = FullNodesGroupMessage::ConfirmGroup(confirm_data);
        return Some((grp_msg, self.full_nodes_accepted.clone()));
    }
    [...]
}
```

Thus, when the next `advance_invites` finally executes, the publisher includes all accepted nodes, exceeding the max group size advertised earlier in `PrepareGroup.max_group_size` Consequently, on the client side, `client::handle_confirm_group_message` will reject the confirmation because the group is larger than the promised limit.

© 2025 Otter Audits LLC. All Rights Reserved.

>_ *monad-bft/monad-raptorcast/src/raptorcast_secondary/client.rs* RUST

```rust
fn handle_confirm_group_message(&mut self, confirm_msg: ConfirmGroup<ST>) {
    [...]
    let confirm_group_size = confirm_msg.peers.len();
    if confirm_group_size > confirm_msg.prepare.max_group_size {
        warn!(
            "RaptorCastSecondary ignoring ConfirmGroup that \
                            is larger ({}) than the promised max_group_size ({}). \
                            Message details: {:?}",
            confirm_msg.peers.len(),
            confirm_msg.prepare.max_group_size,
            confirm_msg
        );
        return;
    }
    [...]
}
```

Remediation

Explicitly include only the first `max_group_size` of the `full_nodes_accepted` list when constructing the `ConfirmGroup` message.

Patch

Resolved in PR#2652.

© 2025 Otter Audits LLC. All Rights Reserved.

Single-Peer BlockSync Liveness Logic MEDIUM OS-MCS-ADV-02

Description

The current BlockSync implementation selects only a single peer via `pick_peer` when fetching missing headers or payloads. Consequently, if that peer is slow or malicious, the leader may hit a timeout before it can fetch the required block. MonadBFT's liveness model assumes bounded-time retrieval. This is significantly weaker than MonadBFT's block-recovery model, which relies on parallel requests to multiple validators to guarantee bounded-time retrieval even in the presence of malicious or slow peers. By depending on one peer, the leader becomes vulnerable to withholding, delay, or selective responsiveness from that peer, enabling an adversarial peer to stall block assembly and trigger unnecessary view changes.

Also, the fixed timeout formula `3 * delta + vote_delay + local_processing` in `get_round_timer` implicitly assumes timely BlockSync completion. When BlockSync is bottlenecked on one peer, this timeout becomes brittle and honest leaders may hit view timeouts even in healthy networks. However, because the pacemaker will eventually increase rounds and regain liveness, the impact is largely limited to edge cases.

Remediation

Synchronize with multiple peers simultaneously. While this results in a trade-off between network bandwidth consumed on synchronization and the expected reliability (reduced latency) of fetching blocks successfully, however, it significantly increases success rates by consulting even a few additional validators.

Patch

The Monad team decides to accept this drawback, stating that malicious validators should be rare, and that the issue will likely resolve itself quickly once honest validators are chosen.

© 2025 Otter Audits LLC. All Rights Reserved.

Risk of Denial-of-Service via Validator Collusion MEDIUM OS-MCS-ADV-03

Description

`client::validate_prepare_group_message` limits the number of groups a full node may participate in utilizing `max_num_group`. Thus, malicious validators may collude by sending overlapping invites that quickly fill a node's allowed slots. Once the limit is reached, the full node will reject any additional invites, including those from honest validators. This effectively blocks honest groups from forming, reducing network connectivity and creating a denial-of-service scenario.

>_ *monad-bft/monad-raptorcast/src/raptorcast_secondary/client.rs* RUST

```rust
fn validate_prepare_group_message(&self, invite_msg: &PrepareGroup<ST>) -> bool {
    [...]
    // Check confirmed groups
    for group in self
        .confirmed_groups
        .values(invite_msg.start_round..invite_msg.end_round)
    {
        [...]
        // Check that it doesn't exceed max number of groups during round span
        num_current_groups += 1;
        if num_current_groups + 1 > self.config.max_num_group {
            log_exceed_max_num_group();
            return false;
        }
    }
    [...]
}
```

Remediation

Utilize a larger limit, such that at least one honest validator may be included.

Patch

The Monad team decides to accept this drawback, stating that full nodes can increase their local `max_num_group` config if they are having trouble syncing.

© 2025 Otter Audits LLC. All Rights Reserved.

Range-Adjacency Check Off-By-One LOW OS-MCS-ADV-04

Description

`assigner::reorder_to_gso` routine tries to merge adjacent chunk-id ranges for the same recipient so they may be emitted as larger, GSO-friendly contiguous slices. However, the current adjacency check utilizes `last.end + 1 == chunk_slice.chunk_id_range.start` (`last.end + 1 == next.start`) , which is incorrect because Rust ranges are half-open ([start, end)), implying two ranges are truly contiguous when `last.end == next.start`.

>_ *monad-bft/monad-raptorcast/src/packet/assigner.rs* RUST

```rust
fn reorder_to_gso(
    chunk_slices: Vec<ChunkSlice<'a, PT>>,
    hint_num_recipients: Option<usize>,
) -> Vec<ChunkSlice<'a, PT>> {
    [...]
    for chunk_slice in chunk_slices {
        let recipient = chunk_slice.recipient;
        let key = recipient.node_hash();
        match messages.entry(key) {
            [...]
            Entry::Occupied(mut e) => {
                let (_recipient, ranges) = e.get_mut();
                let last = ranges.last_mut().expect("occupied entry never empty");
                if last.end + 1 == chunk_slice.chunk_id_range.start {
                    last.end = chunk_slice.chunk_id_range.end;
                    continue;
                }
                out_slices_count += 1;
                ranges.push(chunk_slice.chunk_id_range);
            }
        }
    }
    [...]
}
```

Remediation

Rectify the condition to `last.end == chunk_slice.chunk_id_range.start` correctly merge contiguous ranges.

Patch

Resolved in PR#2739.

© 2025 Otter Audits LLC. All Rights Reserved.

Incorrect Group Expiration Check LOW OS-MCS-ADV-05

Description

The cleanup logic in `publisher::enter_round` incorrectly utilizes `end_round >= new_round`, even though `end_round` is defined as an exclusive boundary. As a result, any group whose `end_round` equals the current round is mistakenly retained even though it has already expired. This results in stale groups remaining in `group_schedule` longer than intended.

>_ *monad-bft/monad-raptorcast/src/raptorcast_secondary/publisher.rs* RUST

```rust
fn enter_round(&mut self, new_round: Round) {
    trace!(?new_round, "enter_round");
    assert_ne!(new_round, Round::MAX);

    if new_round < self.curr_group.get_round_span().end {
        // We don't need to advance to the next group yet
        assert!(new_round >= self.curr_group.get_round_span().start);
        return;
    }

    // Remove all groups that have ended.
    self.group_schedule
        .retain(|_, group| group.end_round >= new_round);
   [...]
}
```

Remediation

Change the condition to `end_round > new_round` to removes all groups whose valid span has ended.

Patch

Resolved in a91136c.

© 2025 Otter Audits LLC. All Rights Reserved.

Incomplete cache eviction LOW OS-MCS-ADV-06

Description

`insert` evicts entries based on count or expiration, not the actual byte size of messages. In the expired-key path, the code removes all expired entries and then assumes this is sufficient to bring the cache under both limits. Similarly, the random-eviction fallback removes exactly one arbitrary entry, regardless of its size, which may be smaller than the newly inserted decoder state. The impact in this case will be limited, as the per-source restrictions prevent attackers from executing large-scale attacks against the cache.

Remediation

Evict cache until it goes below the max capacity.

Patch

Resolved in a5f5d59.

© 2025 Otter Audits LLC. All Rights Reserved.

Improper Inactivated Symbol Handling LOW OS-MCS-ADV-07

Description

`decode_inactive_gaussian::try_inactive_gaussian` returns false when it detects an inactivated symbol of weight one, even though this situation represents an immediate opportunity to reactivate and advance decoding. Returning false misleads the caller into thinking no progress is possible.

>_ monad-bft/monad-raptor/src/r10/nonsystematic/decoder/decode_inactive_gaussian.rs RUST

```rust
pub fn try_inactive_gaussian(
    &mut self,
    xor_buffers: &mut impl FnMut(BufferId, BufferId),
) -> bool {
    [...]
    if self.buffers_inactivated.peek_min().unwrap().1.get() == 1 {
        // There is an inactivated intermediate symbol we can reactivate, so there is no
        // need to perform Gaussian elimination at this point.
        return false;
    }
    [...]
}
```

Remediation

Return true to skip inactivating the symbol if Gaussian observes that there exists a re-activeable symbol.

Patch

Resolved in PR#2740.

© 2025 Otter Audits LLC. All Rights Reserved.

Epoch Boundary Timing Inconsistency LOW OS-MCS-ADV-08

Description

Epoch starting points are determined by the `block.round` of the epoch boundary block (a block whose sequence satisfies `block.sequence % epoch_length = epoch_length - 1`), and the `epoch_start_delay`, measured in rounds. The epoch start round is derived from the proposed round of the epoch-boundary block (`block_round + epoch_start_delay`), but the validator set transition is only learned by the system once that block is actually committed. If commitment is significantly delayed, the logical epoch transition point may already lie in the past by the time the block is finalized.

If the sum of `epoch_boundary_block.round` and `epoch_start_delay` is less than `commit_round`, the new epoch is already active according to round-based logic, yet the new validator set has had no advance notice. Thus, new validators may not have enough time to prepare and start working, During the delay, blocks continue to be produced utilizing the old validator set, yet once the boundary block is committed, retroactive inspection of the past rounds would appear to use the incorrect (old) validator set instead of the correct (new) one. This creates confusion, and possibly even safety and liveness concerns due to validator set uncertainty.

Although `epoch_start_delay` is expected to be large, rendering the likelihood of this scenario extremely low, its existence implies the design is not provably secure.

Patch

The Monad team acknowledges the epoch change scheme is currently only probabilistically secure, but decides to accept the risk since the likelihood of the scenario happening is extremely low, and can be easily detected before an attack may happen.

© 2025 Otter Audits LLC. All Rights Reserved.

04 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MCS-SUG-00	A malicious validator may craft a `TimeoutMessage` with a current-round QC in `high_extend`.
OS-MCS-SUG-01	Dataplane drops messages under congestion, enabling attackers to flood channels and create a denial-of-service scenario.
OS-MCS-SUG-02	The optional block-header fields are encoded without explicit presence markers, so missing earlier fields may result in later values to be misaligned and decoded into the wrong slots.
OS-MCS-SUG-03	A brief TOCTOU window may result in TCP messages to be dropped when a connection task times out before new messages are pushed, leaving them unsent in a newly created channel.
OS-MCS-SUG-04	Recommendation for refactoring the codebase for better functionality and addressing inconsistencies.

© 2025 Otter Audits LLC. All Rights Reserved.

Validate TimeoutMessage with same round QC

OS-MCS-SUG-00

Description

The vulnerability occurs in `TimeoutMessage::validate` because there is no check enforcing that `timeout.tminfo.round < timeout.tminfo.high_qc_round`. A malicious validator may include a QC from the current round in `high_extend`, which passes `verify_high_extend` and the rank check. This allows the creation of a valid timeout message where `timeout.tminfo.round` is equal to `timeout.tminfo.high_qc_round`.

>_ *monad-consensus/src/validation/signing.rs* RUST

```rust
/// A valid timeout message is well-formed, and carries valid QC/TC
pub fn validate<VTF, VT, LT>([...]) -> Result<(), Error>
where
    VTF: ValidatorSetTypeFactory<ValidatorSetType = VT>,
    VT: ValidatorSetType<NodeIdPubKey = SCT::NodeIdPubKey>,
    LT: LeaderElection<NodeIdPubKey = SCT::NodeIdPubKey>,
{
    [...]
    verify_high_extend(
        cert_cache,
        &|epoch, round| {
            epoch_to_validators(epoch_manager, val_epoch_map, election, epoch, round)
        },
        &timeout.high_extend.clone().into(),
    )?;
    if timeout.high_extend.rank() != timeout.tminfo.rank() {
        return Err(Error::NotWellFormed);
    }
    [...]
}
```

While this doesn't have any immediate impacts, such messages should never be created by any correct validators, and could be filtered out during message verification.

Remediation

Enforce `timeout.tminfo.round < timeout.tminfo.high_qc_round` in `TimeoutMessage::validate`.

© 2025 Otter Audits LLC. All Rights Reserved.

UDP/TCP Channel Congestion

OS-MCS-SUG-01

Description

Monad's dataplane drops messages whenever channels or buffers are full, resulting in packet loss. This behavior affects both inbound and outbound messages, which may disrupt validator processing and network reliability. Attackers may exploit this by flooding UDP/TCP channels, saturating inbound message queues and creating a denial-of-service scenario for validators. Currently, mitigation relies solely on external firewalls.

Remediation

Ensure the firewall is properly configured.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Optional-Field Discriminators

OS-MCS-SUG-02

Description

The current encoding omits explicit markers for optional fields, relying solely on payload length to infer whether `base_fee`, `base_fee_trend`, and `base_fee_moment` exist. This creates positional ambiguity. if earlier optional fields are `None` but later ones are `Some`, the decoder will misinterpret the remaining values and assign them to the wrong fields. For example, if a `ConsensusBlockHeader` includes only `base_fee_trend` and not `base_fee`, the current encoding and decoding logic incorrectly interprets the `base_fee_trend` value as `base_fee`.

Remediation

Adding explicit discriminators to differentiate between `None` and `Some`.

© 2025 Otter Audits LLC. All Rights Reserved.

TCP Message Drop via TOCTOU

OS-MCS-SUG-03

Description

There is a TOCTOU (Time-of-Check to Time-of-Use) window in `tx` where `connect_and_send_messages` stops polling the `msg_receiver` after a `MSG_WAIT_TIMEOUT`. If no message arrives within that timeout, the task exits, ending the TCP connection for that peer. Meanwhile, new messages may be pushed to the peer channel via `TxState::push`.

>_ *monad-bft/monad-dataplane/src/tcp/tx.rs* RUST

```rust
async fn connect_and_send_messages(
    conn_id: u64,
    addr: &SocketAddr,
    msg_receiver: &mut mpsc::Receiver<TcpMsg>,
) -> Result<(), Error> {
    [...]
    loop {
        let msg = match msg_receiver.try_recv() {
            Ok(msg) => msg,
            Err(TryRecvError::Disconnected) => break,
            Err(TryRecvError::Empty) => {
                conn_cork(stream.as_raw_fd(), false);

                match timeout(MSG_WAIT_TIMEOUT, msg_receiver.recv()).await {
                    Ok(None) => break,
                    Ok(Some(msg)) => {
                        conn_cork(stream.as_raw_fd(), true);
                        msg
                    }
                    Err(_) => break,
                }
            }
        };
        [...]
    }
    Ok(())
}
```

Messages pushed during the window where receiver task yields on the `sleep` will then be silently dropped without any warnings.

>_ *monad-bft/monad-dataplane/src/tcp/tx.rs* RUST

```rust
async fn task_connection(
    conn_id: u64,
    addr: SocketAddr,
    mut msg_receiver: mpsc::Receiver<TcpMsg>,
```

© 2025 Otter Audits LLC. All Rights Reserved.

```
    _tx_state_peer_handle: TxStatePeerHandle,
) {
    [...]

    if let Err(err) = connect_and_send_messages(conn_id, &addr, &mut msg_receiver).await {
        let mut additional_messages_dropped = 0;

        // Throw away (and fail) all remaining queued messages immediately.
        while let Ok(_msg) = msg_receiver.try_recv() {
            additional_messages_dropped += 1;
        }

        warn!(
            conn_id,
            ?addr,
            ?err,
            additional_messages_dropped,
            "error transmitting tcp message"
        );

        // Sleep to avoid reconnecting too soon.
        sleep(TCP_FAILURE_LINGER_WAIT).await;
    }

    [...]
}
```

Remediation

Add a flag to toggle refusal of new messages before sleep.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-MCS-SUG-04

Description

1. The early return in `publisher::step_until` results in only the first ready group to be processed, leaving other eligible groups unadvanced even tough they are ready at the same time point. This introduces avoidable delays and unfair scheduling. Process all `groups_schedules` and return an aggregated result.

>_ monad-raptorcast/src/raptorcast_secondary/publisher.rs RUST

```rust
fn step_until(
    &mut self,
    time_point: TimePoint,
) -> Option<(FullNodesGroupMessage<ST>, FullNodesST<ST>)> {
    [...]
    // Check if scheduled groups need servicing: time-outs, invites, confirm
    for (_, group) in self.group_schedule.iter_mut() {
        if let Some(out_msg) =
            group.advance_invites(time_point, &self.scheduling_cfg,
                ↪  self.validator_node_id)
        {
            [...]
            // record number of invites
            if let FullNodesGroupMessage::PrepareGroup(_) = &out_msg.0 {
                self.metrics[PUBLISHER_SENT_INVITES] += out_msg.1.list.len() as u64;
            }
            return Some(out_msg);
        }
    }
    [...]
}
```

2. Remove the deprecated Group `cmp` / `partial_cmp` to improve code clarity.
3. The current `range((..)..(..))` bounds in `disconnect_ip` and `disconnect_socket` create a half-open interval, which excludes the upper boundary. To iterate over all entries whose keys fall within the full inclusive range of (`ip`, `u16::MIN`, `u64::MIN`) through (`ip`, `u16::MAX`, `u64::MAX`), the correct form is an inclusive range `..=`.

>_ monad-bft/monad-dataplane/src/tcp/mod.rs RUST

```rust
#[allow(unused)]
pub(crate) fn disconnect_ip(&self, ip: IpAddr) {
    [...]
    for (id, tx) in map.range((ip, u16::MIN, u64::MIN)..(ip, u16::MAX, u64::MAX)) {
        let _ = tx.send(TcpControlMsg::Disconnect);
```

© 2025 Otter Audits LLC. All Rights Reserved.

```
        count += 1;
    }
    [...]
}

#[allow(unused)]
pub(crate) fn disconnect_socket(&self, ip: IpAddr, port: u16) {
    [...]
    for (_, tx) in map.range((ip, port, u64::MIN)..(ip, port, u64::MAX)) {
        let _ = tx.send(TcpControlMsg::Disconnect);
        count += 1;
    }
    [...]
}
```

Remediation

Incorporate the above refactors.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

06 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.